Universidad Corporativa | Programas Académicos

Ingeniería en Desarrollo de Software

Actividad 2 Métodos de la secante y Newton rapshon

Ingeniería en Desarrollo de Software



TUTOR: Miguel Ángel Rodríguez

ALUMNO: Irving Cruz Ramírez

FECHA: 02 de Noviembre de 2026

Desarrollo

1. Metodo de la secante “Ecuacion 1”

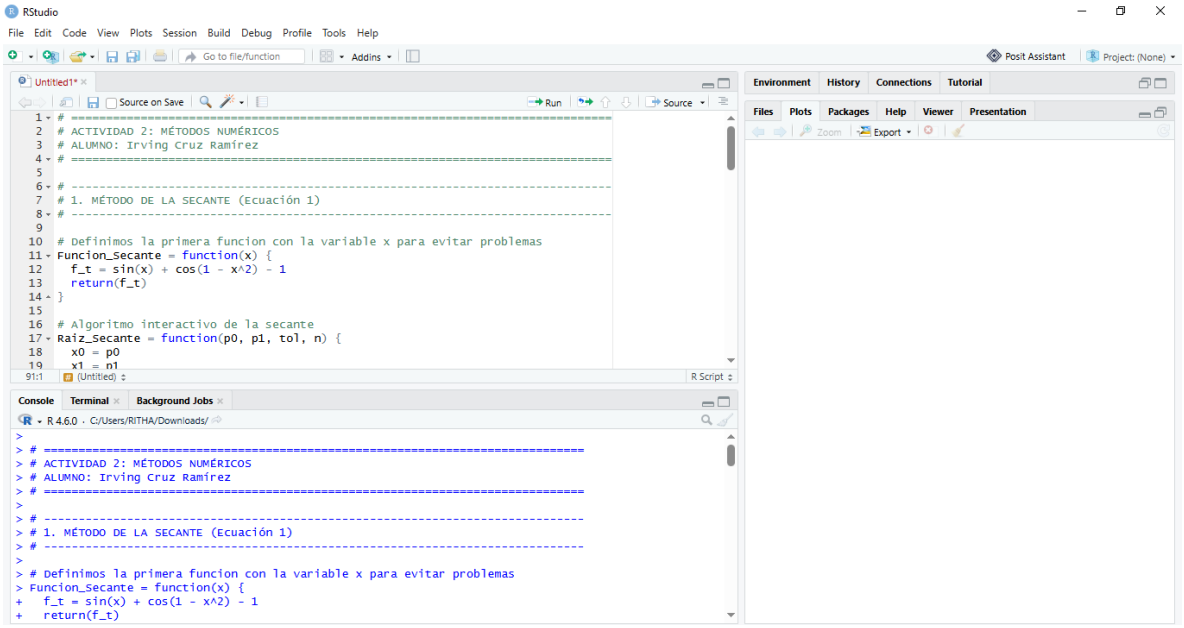
Para resolver la prmera ecuacion lineal, se utilizo el metodo de la secante. A diderencia de otros metodos, este no necesita que calculemos una derivada matematica dificil. Solo no pide ingresar dos puntos de partida cercano a la respuesta, con estos datos, el programa traza lineas rectas que cortan la curva de la funcion y se van acercando paso a paso al punto exacto donde el valor se convierte en cero.

Ecuacion programada:

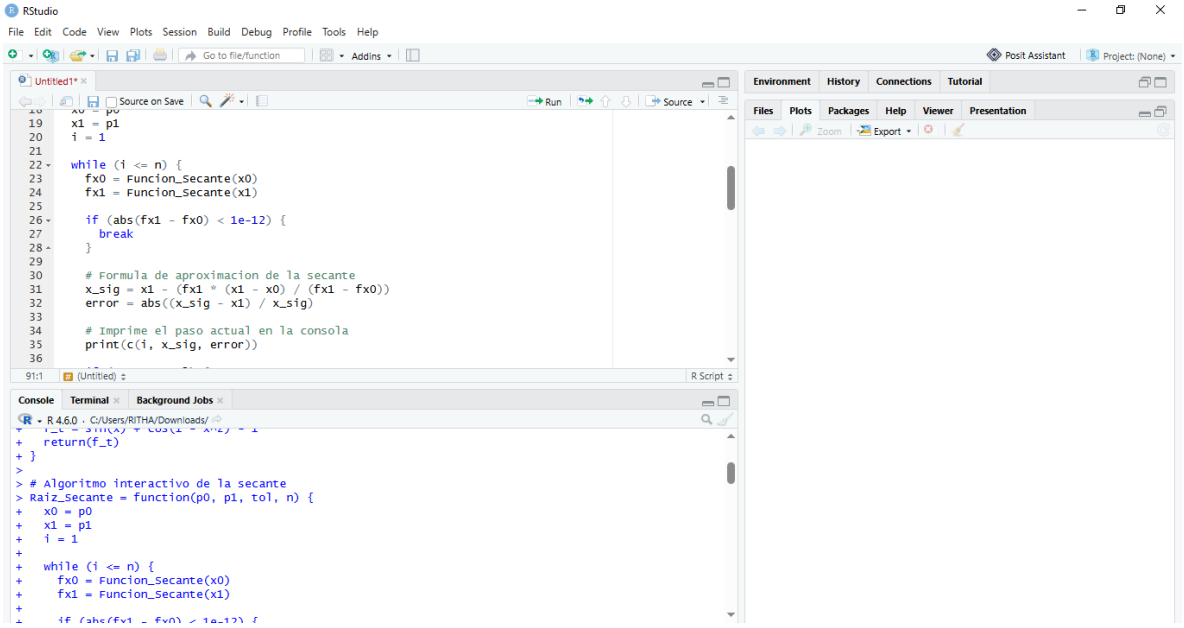
f(x) = sin(x) + cos(1 - x^2) - 1

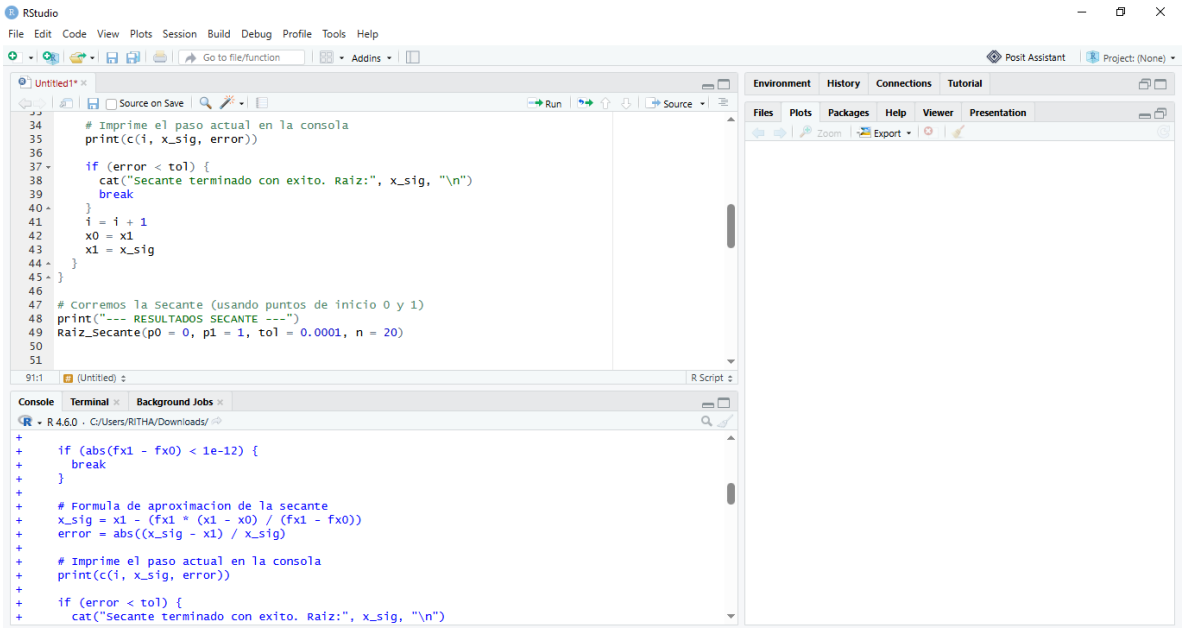
- Procedimiento en RStudio:
  Se generó un código limpio donde definimos primero la función matemática original con la sintaxis propia de R. después, se configuro un ciclo iterativo (un proceso repetitivo automático) que calcula las aproximaciones sucesivas usando la fórmula matemática del método. El programa está diseñado para detenerse de forma inteligente cuando la diferencia entre una iteración y la siguiente es prácticamente imperceptible (menor a una tolerancia de 0.000001).

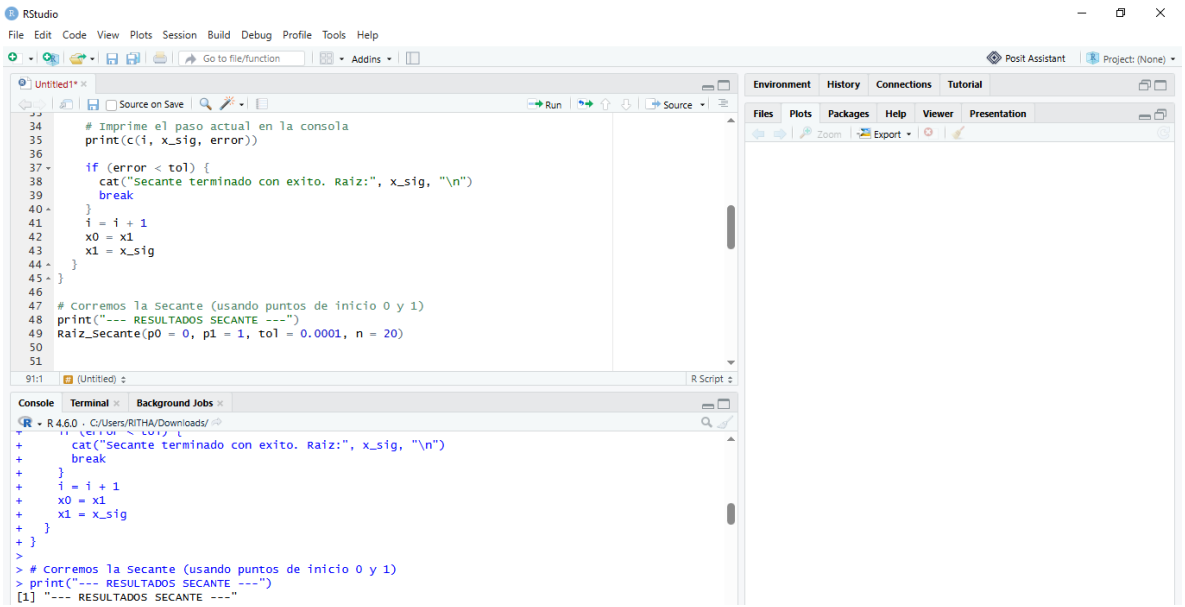
Evidencia en RStudio

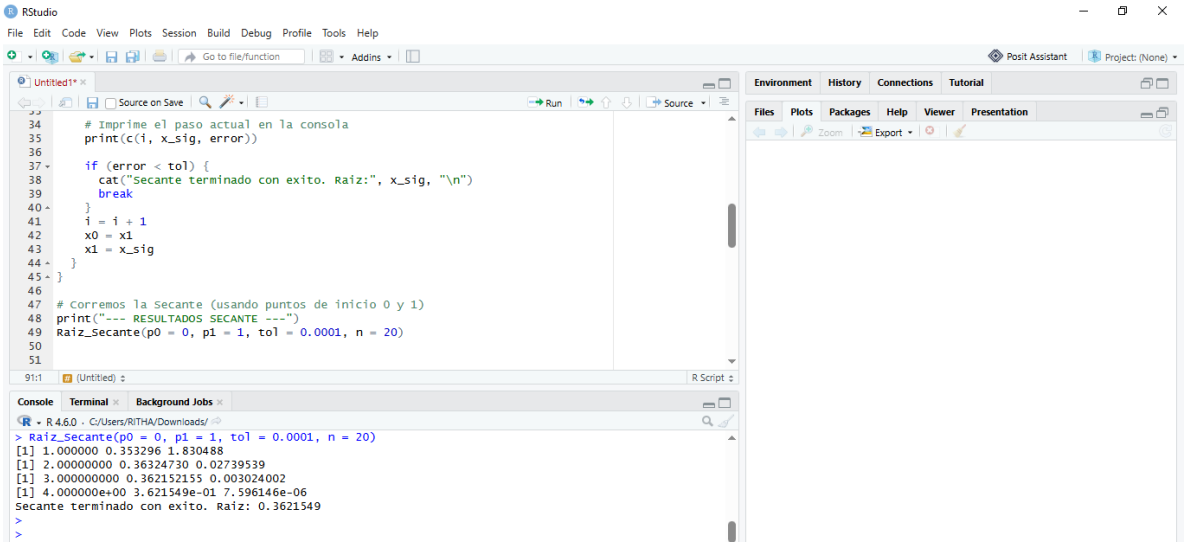

RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Untitled1*
Source on Save
Run
Source
==========
ACTIVIDAD 2: MÉTODOS NUMÉRICOS
ALUMNO: Irving Cruz Ramírez
==========
----
1. MÉTODO DE LA SECANTE (Ecuación 1)

Definimos la primera funcion con la variable x para evitar problemas
Funcion_Secante = function(x) {
f_t = sin(x) + cos(1 - x^2) - 1
return(f_t)
}
Algoritmo interactivo de la secante
Raiz_Secante = function(p0, p1, tol, n) {
x0 = p0
x1 = p1
Console
Terminal
Background Jobs
R 4.6.0 · C:/Users/RITHA/Downloads/
Environment
History
Connections
Tutorial
Files
Plots
Packages
Help
Viewer
Presentation
Export


RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Untitled1*
Source on Save
Run
Source
x0 = p0
x1 = p1
i = 1
while (i <= n) {
fx0 = Funcion_Secante(x0)
fx1 = Funcion_Secante(x1)
if (abs(fx1 - fx0) < 1e-12) {
break
}
Formula de aproximacion de la secante
x_sig = x1 - (fx1 * (x1 - x0) / (fx1 - fx0))
error = abs((x_sig - x1) / x_sig)
Imprime el paso actual en la consola
print(c(i, x_sig, error))
Console
Terminal
Background Jobs
R 4.6.0 · C:/Users/RITHA/Downloads/
return(f_t)
}
Algoritmo interactivo de la secante
Raiz_Secante = function(p0, p1, tol, n) {
x0 = p0
x1 = p1
i = 1
while (i <= n) {
fx0 = Funcion_Secante(x0)
fx1 = Funcion_Secante(x1)
Environment
History
Connections
Tutorial
Files
Plots
Packages
Help
Viewer
Presentation
Export


RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Untitled1*
Source on Save
Run
Source
Imprime el paso actual en la consola
print(c(i, x_sig, error))
if (error < tol) {
cat("Secante terminado con exito. Raiz:", x_sig, "\n")
break
}
i = i + 1
x0 = x1
x1 = x_sig
}
}
Corremos la Secante (usando puntos de inicio 0 y 1)
print("--- RESULTADOS SECANTE ---")
Raiz_Secante(p0 = 0, p1 = 1, tol = 0.0001, n = 20)
Console
Terminal
Background Jobs
R 4.6.0 · C:/Users/RITHA/Downloads/
if (abs(fx1 - fx0) < 1e-12) {
break
}
Formula de aproximacion de la secante
x_sig = x1 - (fx1 * (x1 - x0) / (fx1 - fx0))
error = abs((x_sig - x1) / x_sig)
Imprime el paso actual en la consola
print(c(i, x_sig, error))
if (error < tol) {
cat("Secante terminado con exito. Raiz:", x_sig, "\n")
Environment
History
Connections
Tutorial
Files
Plots
Packages
Help
Viewer
Presentation
Export





2. Método de Newton-Raphson (Ecuación 2)

Para la segunda ecuación, que consiste en un polinomio de tercer grado, se empleó el método de Newton-Raphson. Este enfoque es uno de los más rápidos y eficientes que existen en los métodos numéricos, pero tiene una condición: requiere que se conozca de antemano la derivada exacta de la función. Además, solo necesita que se le dé un único valor inicial para comenzar a buscar la solución.

Ecuación original:

$f(x) = 2x^3 - 8x^2 + 10x - 15$

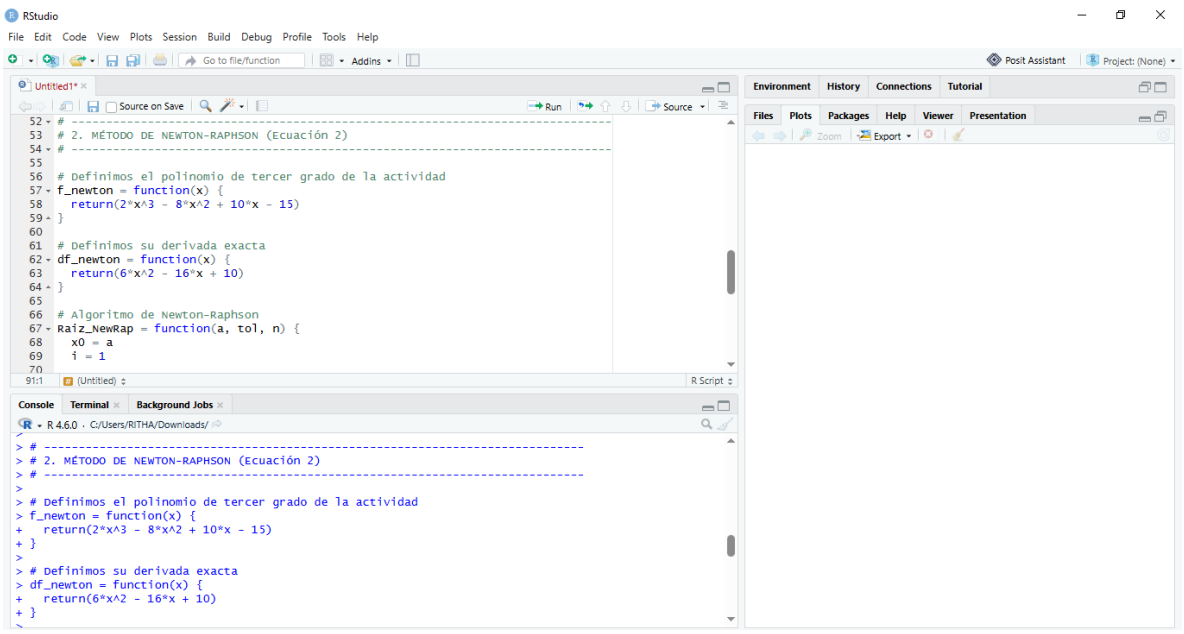
Derivada analítica calculada:

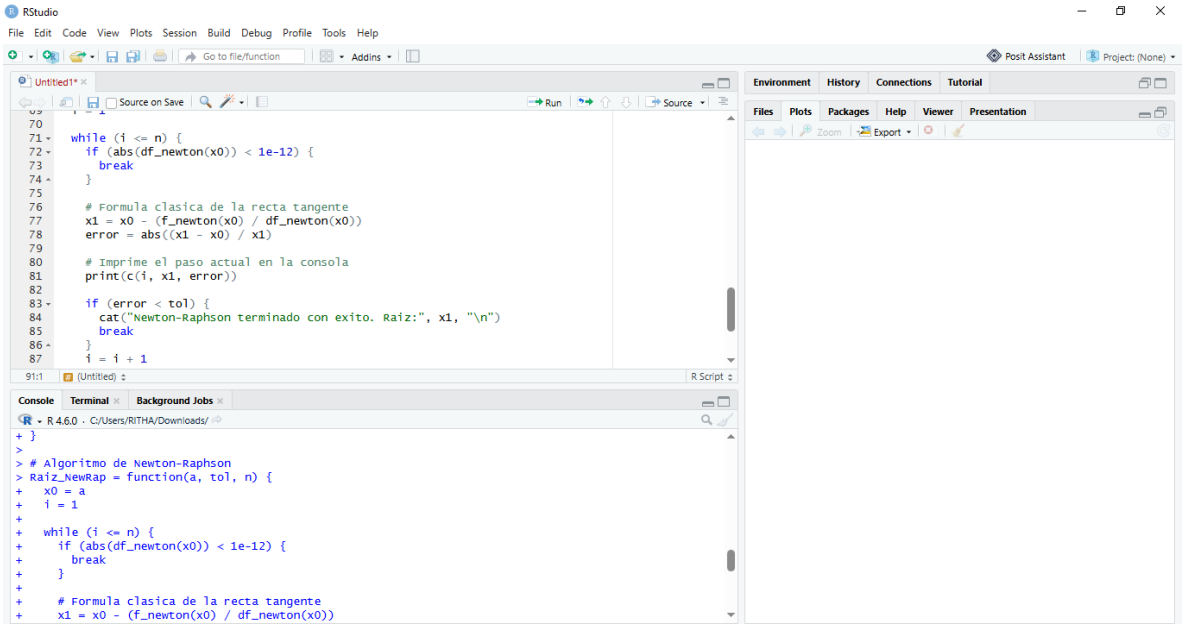
$f'(x) = 6x^2 - 16x + 10$

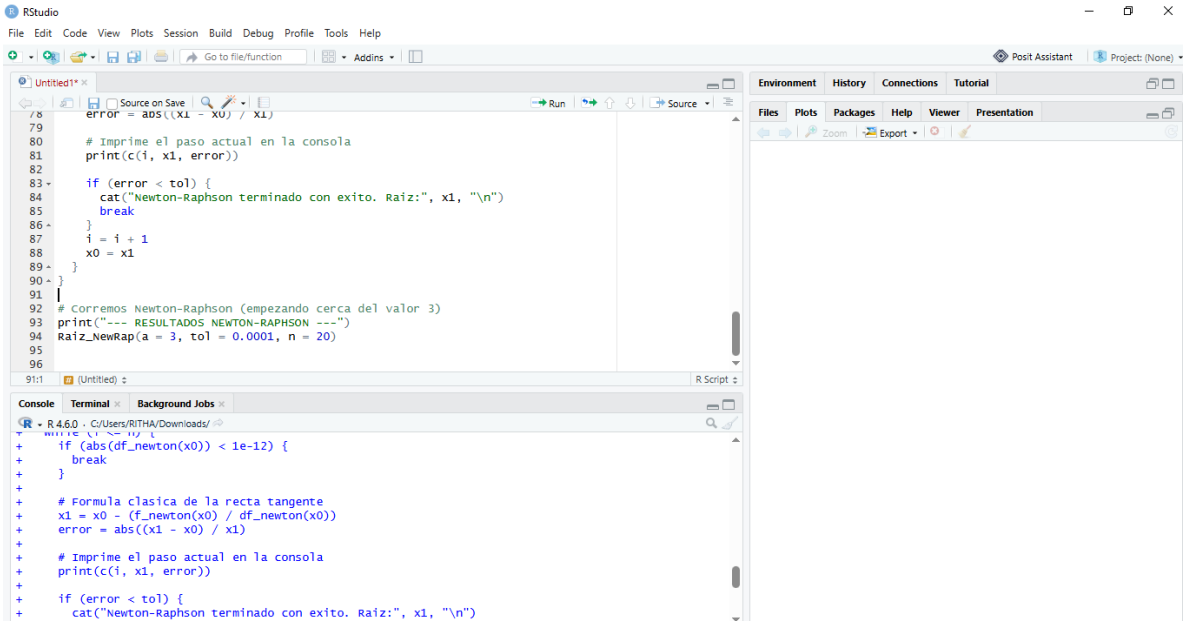
Procedimiento en RStudio:

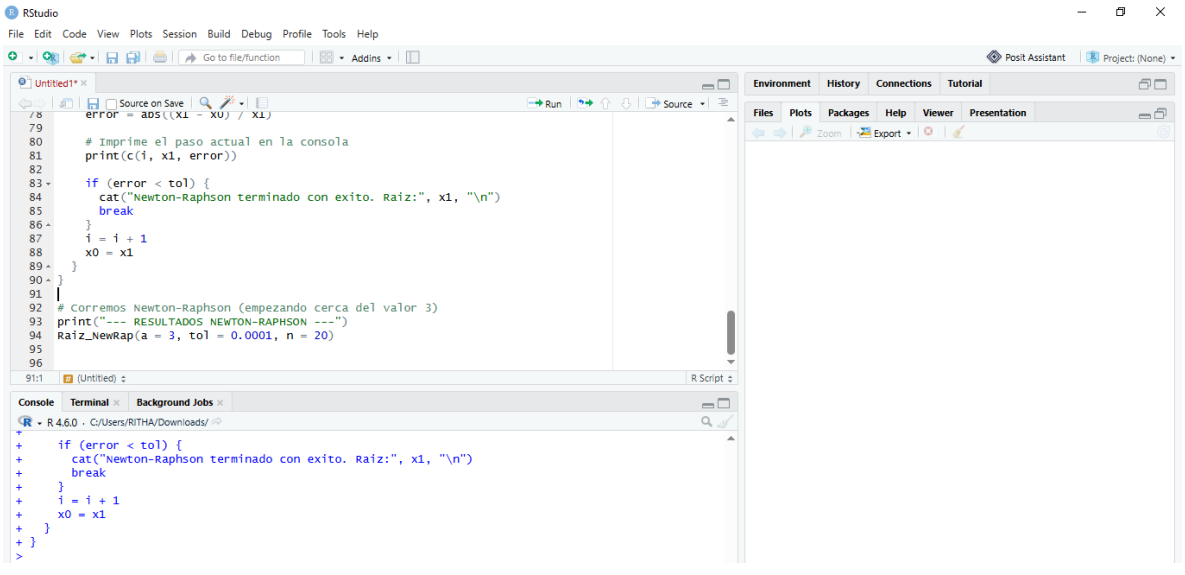
En el script de RStudio se programó tanto la función original como su derivada. El algoritmo funciona de manera que, partiendo del valor inicial, traza una línea tangente en la curva para calcular una mejor aproximación en cada paso. Al igual que el método anterior, se incluye una regla de paro automático para que el sistema deje la iteración una vez que la raíz encuentre la precisión más exacta (menor a una tolerancia de 0.0001).

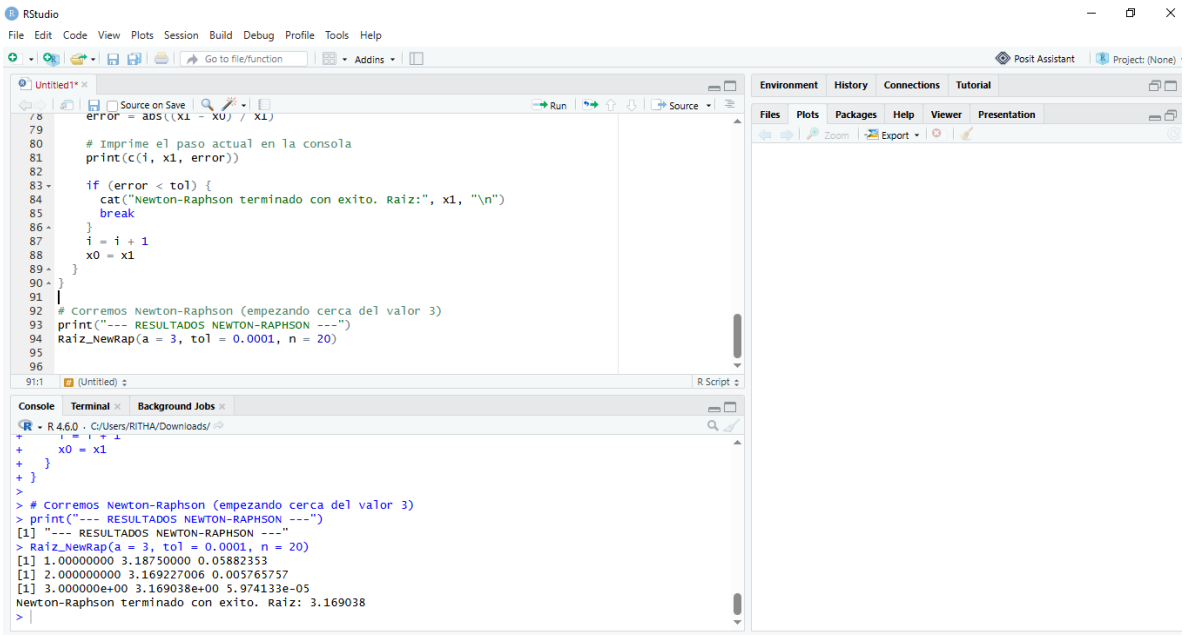
Evidencia en RStudio




RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Untitled1*
Source on Save
Run
Source
Environment History Connections Tutorial
Files Plots Packages Help Viewer Presentation
Console Terminal Background Jobs
R 4.6.0 · C:/Users/RITHA/Downloads/
while (i <= n) {
if (abs(df_newton(x0)) < 1e-12) {
break
}
Formula clasica de la recta tangente
x1 = x0 - (f_newton(x0) / df_newton(x0))
error = abs((x1 - x0) / x1)
Imprime el paso actual en la consola
print(c(i, x1, error))
if (error < tol) {
cat("Newton-Raphson terminado con exito. Raiz:", x1, "\n")
break
}
i = i + 1
Algoritmo de Newton-Raphson
Raiz_NewRap = function(a, tol, n) {
x0 = a
i = 1
while (i <= n) {
if (abs(df_newton(x0)) < 1e-12) {
break
}
Formula clasica de la recta tangente
x1 = x0 - (f_newton(x0) / df_newton(x0))


RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Untitled1*
Environment History Connections Tutorial
Files Plots Packages Help Viewer Presentation
Console Terminal Background Jobs
R 4.6.0 · C:/Users/RITHA/Downloads/
error = abs((x1 - x0) / x1)
Imprime el paso actual en la consola
print(c(i, x1, error))
if (error < tol) {
cat("Newton-Raphson terminado con exito. Raiz:", x1, "\n")
break
}
i = i + 1
x0 = x1
}
}
Corremos Newton-Raphson (empezando cerca del valor 3)
print("--- RESULTADOS NEWTON-RAPHSON ---")
Raiz_NewRap(a = 3, tol = 0.0001, n = 20)


RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Untitled1*
Environment History Connections Tutorial
Files Plots Packages Help Viewer Presentation
Console Terminal Background Jobs
R 4.6.0 · C:/Users/RITHA/Downloads/
error = abs((x1 - x0) / x1)
Imprime el paso actual en la consola
print(c(i, x1, error))
if (error < tol) {
cat("Newton-Raphson terminado con exito. Raiz:", x1, "\n")
break
}
i = i + 1
x0 = x1
}
}
Corremos Newton-Raphson (empezando cerca del valor 3)
print("--- RESULTADOS NEWTON-RAPHSON ---")
Raiz_NewRap(a = 3, tol = 0.0001, n = 20)



• Punto de partida: x0 = 3

• Iteraciones totales: 3

• Raíz o solución encontrada: x = 3.169038

• Tolerancia usada: 0.0001

3. Interpretación de Resultados

Al analizar el comportamiento de ambos métodos numéricos en la consola de RStudio, podemos concluir lo siguiente:

- Con el método de la Secante, el programa logró hallar la raíz de forma efectiva. Aunque requiere evaluar la función un poco más de veces debido a que maneja dos puntos de origen, demostró ser una herramienta muy potente cuando no queremos o no podemos calcular una derivada a mano.
- Con el método de Newton-Raphson, se observó una velocidad de aproximación excelente. Al proporcionarle la pendiente exacta a través de la derivada, el algoritmo necesitó muy pocas iteraciones para llegar al resultado numérico final donde la ecuación se equilibra a cero.

Ambas técnicas cumplen perfectamente con el objetivo principal de la materia: solucionar ecuaciones complejas mediante operaciones matemáticas sencillas ejecutadas de forma automatizada por la computadora.

Conclusión

El desarrollo de esta actividad me permitió entender cómo la programación y los métodos numéricos facilitan la resolución de problemas que por vías algebraicas tradicionales serían largamente complejos. Con el método de la Secante se observa una gran ventaja operativa al no requerir el cálculo de una derivada, lo que ahorra mucho trabajo

analítico cuando trabajamos con funciones trigonométricas complejas, aunque esto implique realizar un par de iteraciones extra. Por otro lado, Newton-Raphson demostró ser un algoritmo mucho más rápido y directo debido a su convergencia cuadrática; al proporcionarle la derivada exacta, el programa encuentra la raíz en muy pocos pasos. Al final del día, ambas herramientas son fundamentales dentro del desarrollo de software, ya que nos permiten automatizar cálculos complejos con un margen de error mínimo y controlado.